ASSIGNMENT-8

Spike-Driven Violence Detection and Automated Evidence Clip Generation

Team members:

23MID0436-K.Hemashwaran

23MID0440-P H Rohan

Faculty: Anny Leema mam

Course Code: MDI4001

Course Title: Machine Learning for Data Science

1. Introduction

For this assignment I am working on a violence detection system for surveillance cameras that uses a Spiking Neural Network (SNN) instead of a normal deep network. The reason for choosing an SNN is power. Cameras installed at the edge run all day, and most of what they see is an empty corridor or a parking lot where nothing happens. An SNN only does work when something changes in the scene, so in principle it should use far less energy than a regular CNN or LSTM model looking at every frame.

The system does not stop at saying “violence detected”. It also decides when the incident started and ended, and saves a short video clip of it along with a metadata file and a hash, so the clip can be used as evidence later. This report explains the design and what I have finished so far.

2. Objectives

- Detect violence in streaming video using a spiking network that keeps its state over time.
- Find the start and end of an incident automatically, without flickering on and off.
- Save an evidence clip with metadata and a SHA-256 hash so it cannot be changed unnoticed.
- Estimate how much energy the SNN saves compared to ANN models of similar size.
- Test how well it works on the same dataset and on a different dataset.

3. System Design

Figure 1 shows how the whole system fits together. Video frames go through preprocessing, then into the spiking network, whose output goes to the Incident Neuron. When the Incident Neuron decides an incident has started or ended, the clip assembler cuts the saved frames and writes the output files.

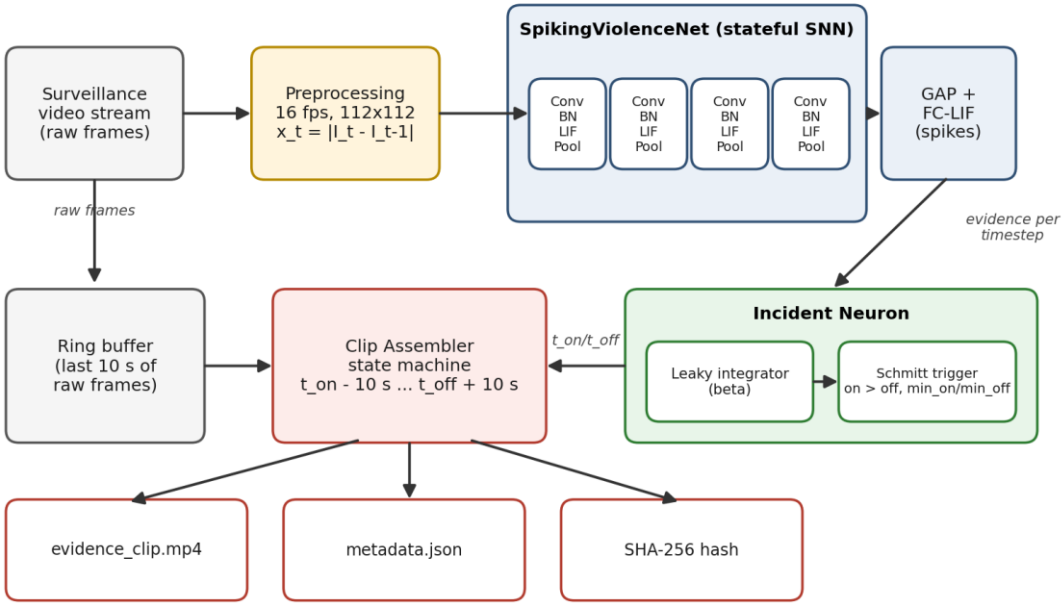


Figure 1. Overall system architecture

3.1 Input: frame difference

Instead of feeding full frames, I give the network the difference between two consecutive frames, $x_t = |I_t - I_{(t-1)}|$. Where nothing moves, the difference is zero, so those pixels produce no spikes. Videos are first reduced from 30 to 16 fps and resized to 112×112.

3.2 Spiking backbone

The network (SpikingViolenceNet) has four blocks of Conv3×3, BatchNorm, LIF neuron and MaxPool, followed by global average pooling and a final LIF layer. The membrane potentials are kept between timesteps, so it can process a video stream frame by frame. It is built with SpikingJelly and works in both single-step and multi-step mode. I also added hooks to record the firing rate of every layer, which is needed later for the energy calculation.

3.3 Incident Neuron

This is the main idea of the project. The output of the SNN is passed through a leaky integrator, and then a Schmitt trigger with two thresholds (θ_on higher than θ_off) decides whether an incident is going on. Two counters (min_on and min_off) make sure a single noisy frame cannot start or end an event. Because of the two thresholds, the decision does not flicker when the score hovers near one value, which happens a lot with a single threshold on a sliding window.

3.4 Evidence clip assembler

A ring buffer always holds the last 10 seconds of raw frames. When an incident starts at t_on and ends at t_off, the assembler writes an MP4 from 10 s before t_on to 10 s after t_off, a JSON file with the details, and a SHA-256 hash of the clip.

3.5 Datasets, baselines and energy model

RWF-2000 will be used for training and testing, and Hockey Fight and RLVS for testing on other data. For comparison I implemented three baselines: MobileNetV2 + GRU, MobileNetV2 + average pooling, and an ANN with the same layout as the SNN. For boundary testing, a script builds long synthetic surveillance streams (10–30 minutes) with violent clips inserted at known times, keeping violence below 5% of the video. Energy is estimated by counting synaptic operations from the measured firing rates and comparing with the MAC count of the ANN models.

4. Work Completed

So far I have finished building the full code base, which has 19 files grouped into seven parts:

Part	Files
Configuration and setup	configs/default.yaml, requirements.txt, configs/colab_setup.py
Data pipeline	download_rwf.py, prep_rwf.py, make_streams.py, dataset.py
Models	snn.py, baselines.py, incident.py
Training	train.py (multiple seeds, resume from checkpoint, TensorBoard)
Evaluation	eval_cls.py, eval_boundary.py, energy.py
Streaming pipeline	assembler.py, stream_runner.py
Documentation	README.md, notebooks/colab_training.ipynb

While testing I found and fixed two bugs in train.py: the SNN returns a tuple (spikes and membrane logits) which the training loop was not handling, and the flag that marks a model as an SNN was wrong for the ANN mirror network.

4.1 Testing

I wrote a test script (test_smoke.py) and ran it on my laptop after installing the required libraries. All 8 tests passed:

No.	Test	What it checks
1	Config	YAML file loads correctly
2	SNN forward pass	Single-step and multi-step inputs work
3	Baselines	All three ANN baselines run
4	Incident Neuron	Leaky integration and hysteresis transitions
5	Clip assembler	Ring buffer and MP4 / JSON / SHA-256 output
6	Energy model	SOP-based energy calculation runs
7	Metrics	Classification and boundary metric functions
8	Stream generator	Synthetic stream and ground-truth files are created

These tests only confirm that the code runs correctly. The model is not trained yet, so the energy value printed in the test (about 0.122 mJ per clip) is just a placeholder and I am not treating it as a result.

5. Remaining Work

The next stage needs a GPU, so I plan to run it on Google Colab. The plan is:

Week	Planned work
1	Set up Colab and the Kaggle API, download the three datasets and run preprocessing.
2–3	Train the SNN with 3 random seeds, and train the three baselines the same way.
4	Generate 10–20 synthetic streams and tune the Incident Neuron parameters (β from 0.8 to 0.95, θ_on from 0.5 to 0.8, θ_off from 0.2 to 0.4). Measure tIoU, onset delay and false triggers per hour.
5	Test on Hockey Fight, record the layer-wise firing rates, and compare SNN and ANN energy.
6	Collect the results into tables and figures and write up the findings.

6. Current Status

Stage	Status
System design	Done
Code implementation	Done
Local testing (8 of 8 passed)	Done
Dataset download and preprocessing	Not started
Training and evaluation	Not started
Energy analysis and report writing	Not started

7. Conclusion

The design and implementation of the system are complete and the code has passed its basic tests. What remains is the experimental part: training the models on real data, checking how well the Incident Neuron finds event boundaries, and measuring the energy saving. I expect to start this in the coming week.